SOURCE-BASED COLLEGE CAPSTONE GUIDE

THREAT TRACE

Complete Project Study Guide

Java 17 | Spring Boot 3.2.5 | Spring Data JPA | MySQL

Understand the real implementation, follow user requests,
study the schema, and prepare for OOP and DBMS viva questions.

EVIDENCE STANDARD

Claims are tied to checked-in code, SQL, configuration, or dump.
Design notes and snapshot data are distinguished from live state.

Prepared from repository review; direct database state was not queried.

Contents

Select a section title to jump. Page numbers are generated from the final layout.

Contents 2
1. Project overview and boundaries 3
2. Complete user flow 4
3. API and request/response guide 6
4. Package and file directory guide 7
5. End-to-end traces and code evidence 9
6. OOP and Java concepts with evidence 11
7. Database and DBMS guide 12
8. Errors, security, and known limitations 14
9. OOP viva questions 15
10. DBMS viva questions 16
11. Presentation cheat sheet 17
12. Repository inventory and glossary 18

Study order: read the user flow first, then the code map and OOP/DBMS evidence. Finish with the viva rehearsal.

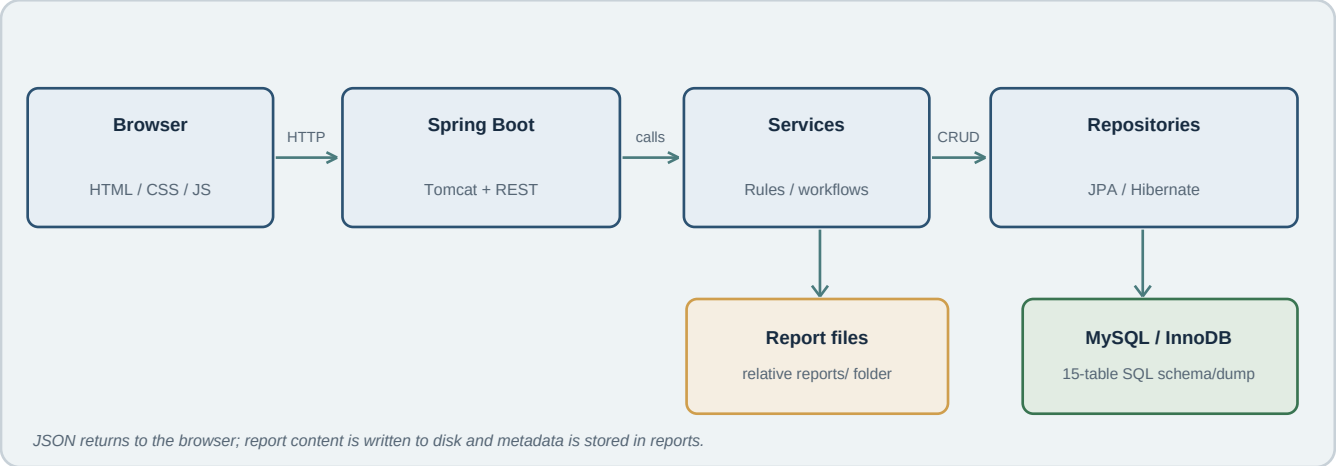
1. Project overview and boundaries

Purpose. Threat Trace is a browser-based investigation tool for creating cases, importing endpoint event records from CSV/JSON, presenting a chronological timeline, reconstructing event parent-child relationships, applying rule-based Cyber Kill Chain mappings, generating a behavioral summary, calculating a risk score, ranking mitigation catalog items, storing analyst notes, and exporting TEXT/JSON/CSV reports. These claims are verified in the UI, REST controllers, services, entities, and repositories.

Problem addressed. The app gathers a set of security events into a case so a student analyst can inspect order, relationships, selected stage matches, and a repeatable prioritization score. Similarity and risk are investigative aids, not proof that a threat or actor is real.

Technology. Java 17; Spring Boot 3.2.5; Spring MVC and embedded Tomcat; Spring Data JPA/Hibernate; MySQL Connector/J; MySQL target; OpenCSV and Jackson; plain HTML/CSS/browser JavaScript; Maven Wrapper. H2 is listed runtime-scope, but no H2 profile or datasource override is configured.

Not implemented in the main app: login/auth enforcement; case edit/close/delete; event edit/delete; PDF export; real-time stream; AI; and analyst identity selection. Timeline refresh uses 15-second HTTP polling only while the Timeline tab is active.



The browser gets JSON responses. JPA repositories use Hibernate for object-relational mapping; Connector/J communicates with MySQL. Report content is also written to a relative reports/ directory.

How startup works

The entry point is src/main/java/com/threattrace/ThreatTraceApplication.java: @SpringBootApplication and main(String[]) call SpringApplication.run. Spring Boot starts embedded Tomcat on configured port 8080. Static file src/main/resources/static/index.html is the welcome page at /. It loads static/css/styles.css and static/js/app.js. No custom root page controller exists.

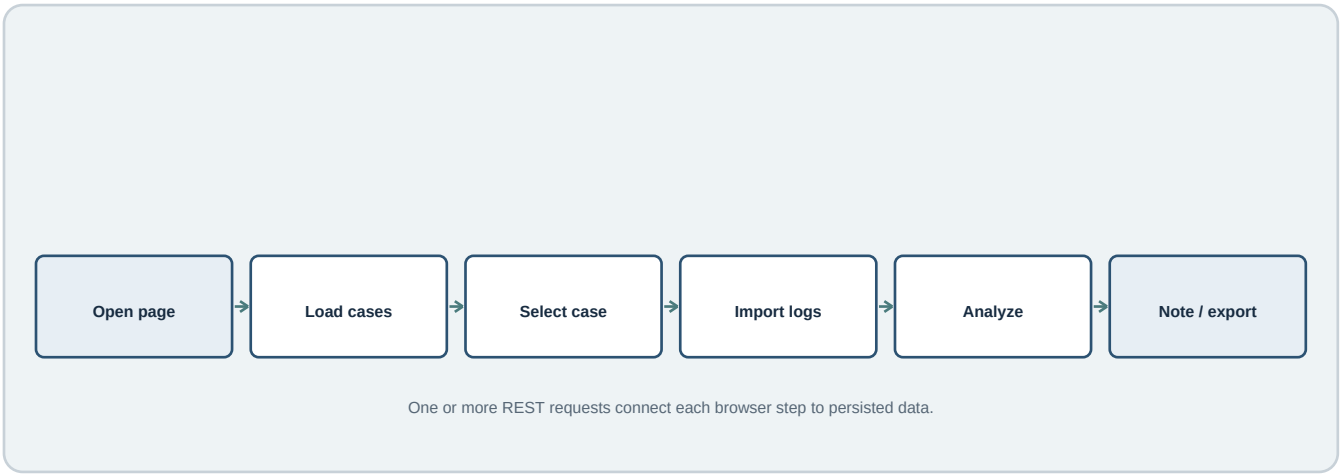
application.properties points to MySQL localhost:3306/threat_trace_db and sets spring.jpa.hibernate.ddl-auto=update. The SQL scripts are not invoked from Java services. schema.sql is a manual script that drops tables before recreating them. Do not run it against data you need.

Implemented vs. described

Area	Implemented in source	Not present / not verified
Case	List/read/create	No case update/close/delete endpoint.
Events	CSV/JSON import, ordered timeline, type/severity/hostname filters, process tree	No UI time range filter, sorting, event edit, or stream.
Analysis	Rule mapping, DNA features, profile similarity, risk, mitigation ranking	No machine-learning attribution or measured mitigation outcome.
Notes/report	Append note and false-positive flag; TEXT/JSON/CSV export	No PDF, note edits, or mitigation acceptance workflow.
Identity	User table/entity and role field	No login endpoint or Spring Security enforcement.

2. Complete user flow

Browser calls use same-origin paths. Controllers return ResponseEntity JSON; Spring/Jackson serializes entities and DTOs. The sequence below is grounded in app.js and controller/service methods.



2.1 Open the page and select a case

```
GET /api/cases
 -> CaseController.getAllCases()
 -> InvestigationCaseRepository.findAll()
 -> app.js selects preferred/current/first case
 -> GET /api/cases/{id}
 -> CaseController.getCaseById()
 -> caseRepository.findById(id)
 -> GET /api/events/timeline/{id}
 -> POST /api/analysis/kill-chain/{id}
 -> GET /api/analysis/dna/{id}
```

DOMContentLoaded calls initializeApp, which starts the interval and loads cases. If none exist, the UI shows an empty-case message. If a request fails, requestJSON creates an Error from a JSON message/error or status text; the relevant panel displays it. Risk, process tree, mitigations, and notes are loaded when their tabs open.

2.2 Create a case

Chain: New case -> openNewCaseModal() -> createNewCase() checks browser-side that case number and title are present -> POST /api/cases with JSON caseNumber/title/description -> CaseController.createCase() -> userRepository.findById(1) -> new InvestigationCase(...) -> caseRepository.save() -> INSERT investigation_cases -> saved entity response -> app reloads case list and selects it. The controller supplies defaults for absent fields. It has no Bean Validation/format checks. A duplicate case_number can fail at the DB unique constraint. If user 1 is absent, analyst assignment is null.

2.3 Import CSV/JSON

Chain: Import log file -> openUploadModal() checks selected case -> uploadLogFile() checks filename extension -> multipart field file to POST /api/events/import/{csv|json}/{caseId} -> EventController -> EventImportService -> EventValidationService -> build EventLog plus device/asset references -> eventLogRepository.saveAll() -> event_logs and possibly endpoint_devices/assets -> ImportResultDTO -> UI shows counts and reloads case/timeline.

CSV headers are trimmed/lowercased. JSON must deserialize as a list of string maps. Validator checks event_code, event_type, event_time, hostname, duplicates, and timestamp parsing. It does not validate allowed enums, field sizes, host/IP format, or full JSON schema. The app config sets a 10 MB file and request limit. UI checks extension, but no content-type or extension validation is in the endpoint.

Bad rows are collected; valid rows are still saved. Import exceptions are often caught and returned inside ImportResultDTO, normally HTTP 200. The UI displays processed/valid/error counts but not validationErrors details. A missing code/hostname is rejected even though builder has a UUID/UNKNOWN-HOST fallback. Unknown hostname may create a device with placeholder IP/OS and asset id 3 or a new default asset. Parent resolution is one pass: earlier batch row, then database event_code, then numeric ID; later rows are not resolved in a second pass.

2.4 Live timeline

loadTimeline sends GET /api/events/timeline/{caseId}; TimelineBuilder reads ordered events and applies optional filters in Java. app.js refreshes every 15 seconds while a case is selected and Timeline is active; Refresh timeline calls the same endpoint. The UI filters client-side after receiving all records. Its hostname match is substring-based; service hostname filtering is exact-match. No WebSocket or server push exists.

2.5 Process lineage

Process lineage tab -> GET /api/events/process-tree/{caseId} -> ProcessLineageBuilder finds null-parent roots and recursively fetches child events -> ProcessNodeDTO forest -> frontend recursively creates DOM cards. A path-local Set detects re-entered event IDs; depth above 25 throws CycleDetectedException. This is Java recursion, not the SQL CTE. There is no global controller exception handler.

2.6 Kill Chain and DNA

Frontend sends POST /api/analysis/kill-chain/{caseId}; KillChainMapper reads events/stages, applies process/target/payload/network rules, and saves EventStageMapping rows. Then GET /api/analysis/dna/{caseId} reads events, mappings, and all profiles. DNA returns event-type sequence, time gaps, hostnames, stages, lead match, and similarity. The similarity calculation only compares signature stage names with observed stage names using case-insensitive substring matching. It does not use returned time gaps or hosts in the score. Existing mapping for an event causes the mapper to skip that event on later runs.

2.7 Risk, recommendations, notes, exports

Risk: POST /api/analysis/risk/{caseId} -> RiskCalculator loads case/events/mappings, scores, updates case risk/severity, saves RiskAssessment JSON and returns breakdown. Empty-event case stores score 0 on the case but returns before inserting an assessment. **Mitigations:** GET /api/analysis/mitigations/{id}?strategy=... chooses a strategy and inserts/updates case_mitigations, then returns descending scores. “Expected effectiveness” is a ranking formula, not measured treatment outcome.

Notes: browser checks non-empty text -> POST /api/analysis/notes/{id} JSON noteText/isFalsePositive -> controller passes analyst id 1 -> transactional service saves AnalystNote and AuditLog -> response -> UI reloads notes. **Export:** POST /api/reports/export/{id}?format=... reads case/events, builds tree, recalculates risk, computes DNA, writes file under reports/, inserts Report row. Browser downloads via GET /api/reports/download?filePath=... . TEXT, JSON, and CSV contain different subsets; notes and mitigation decisions are not included.

3. API and request/response guide

There are 16 mapped controller operations across four controllers (fourteen in the case/event/analysis groups and two report operations). Only GET and POST mappings are present; no PUT, PATCH, DELETE mapping was found.

Method / path	Controller method	Input -> response / persistence
GET /api/cases	CaseController.getAllCases	None -> list cases; SELECT investigation_cases.
GET /api/cases/{id}	getCaseById	Long ID -> case; 404 when missing.
POST /api/cases	createCase	JSON caseNumber/title/description -> saved case; INSERT.
POST /api/events/import/csv/{caseId}	EventController.importCsv	multipart file -> ImportResultDTO; event/device/asset writes.
POST /api/events/import/json/{caseId}	importJson	multipart file -> ImportResultDTO; same writes.
GET /api/events/timeline/{caseId}	getTimeline	Optional eventType/severity/hostname -> ordered event list.
GET /api/events/process-tree/{caseId}	getProcessTree	Long ID -> recursive ProcessNodeDTO forest.
POST /api/analysis/kill-chain/{caseId}	AnalysisController.mapKillChain	Long ID -> mappings; may insert event_stage_mappings.
GET /api/analysis/dna/{caseId}	getAttackDNA	Long ID -> AttackDNASummaryDTO.
POST /api/analysis/risk/{caseId}	calculateRisk	Long ID -> RiskScoreBreakdownDTO; updates case and assessment.
GET /api/analysis/mitigations/{caseId}	getMitigations	Optional strategy, default MAX_RISK_REDUCTION -> case mitigation rows.
POST /api/analysis/notes/{caseId}	addNote	JSON noteText/isFalsePositive; analyst fixed to 1; writes note and audit.
GET /api/analysis/notes/{caseId}	getNotes	Long ID -> newest-first notes.
POST /api/reports/export/{caseId}	ReportController.exportReport	Optional userId default 1, format default TEXT -> report row and file.
GET /api/reports/download?filePath=...	downloadReport	Required path -> FileSystemResource; 404 if missing; path not scoped.

Most controller methods return ResponseEntity.ok. GET case and download explicitly return 404 when absent. No @RestControllerAdvice/global error handler was found. Import validation failures often return a result DTO with HTTP 200. The UI helper requestJSON surfaces non-2xx messages.

4. Package and file directory guide

Java packages match directory paths. For example, package com.threattrace.service in RiskCalculator.java lets another class import com.threattrace.service.RiskCalculator. Packages group code and provide a namespace.

```
src/main/resources/static/  index.html, css/styles.css, js/app.js
src/main/java/com/threattrace/
  ThreatTraceApplication.java
  controller/  CaseController, EventController, AnalysisController, ReportController
  service/     import, validation, timeline, lineage, mapping, DNA, risk,
               mitigation, note, report workflows
  service/strategy/ MitigationRankingStrategy + 3 implementations
  domain/      15 JPA entity classes
  repository/  15 Spring Data repository interfaces
  dto/DTOs.java; exception/CustomExceptions.java
src/main/resources/ application.properties; db/schema.sql, seed.sql,
                    recursive_queries.sql; static UI files
src/test/java/... 3 unit test classes
sample-data/ 5 CSV/JSON examples; docs/ 5 project notes
concept-demos/ optional C++ and Python tools
threat_trace_db.sql saved SQL snapshot
```

Significant files and dependencies

Path	Purpose; what calls it / what it calls
ThreatTraceApplication.java	Boot entrypoint; Spring scans controller/service/component packages.
application.properties	Port, MySQL URL, Hibernate ddl-auto, upload limits, logging; secret intentionally not repeated.
static/index.html	Buttons, modals, tabs and response target IDs. Loads app.js and CSS.
static/js/app.js	Frontend controller; fetch endpoints, renders content, handles cases/uploads/analysis.
static/css/styles.css	Theme, responsive layout, evidence table, panels.
controller/CaseController.java	Case GET/create API; calls repositories directly.
controller/EventController.java	Import/timeline/tree API; delegates to services.
controller/AnalysisController.java	Mapping/DNA/risk/mitigation/note API; delegates to services.
controller/ReportController.java	Export/download API; delegates to ReportService / file resource.
service/EventImportService.java	Parses rows, uses validator and repos, builds EventLog/device/asset entities.
service/EventValidationService.java	Required fields, duplicate checks and timestamp parsing.
service/TimelineBuilder.java	Ordered event query, optional in-memory filters.
service/ProcessLineageBuilder.java	Root lookup, recursive child lookup, DTO forest.
service/KillChainMapper.java	Rule matching and mapping writes.
service/AttackDNAAnalyzer.java	Features and profile signature similarity.
service/RiskCalculator.java	Risk formula, case update, assessment record.
service/MitigationAdvisor.java + service/strategy/*.java	Strategy selection, ranking and persistent recommendation rows.
service/AnalystNoteService.java	Writes notes and audit row transactionally.
service/ReportService.java	Builds report contents, writes file, stores metadata.
domain/*.java	Entity fields/relationships; loaded by repositories and used by services.
repository/*.java	Spring Data derived queries and inherited find/save methods.
dto/DTOs.java	ProcessNodeDTO, RiskScoreBreakdownDTO, AttackDNASummaryDTO, ImportResultDTO.
exception/CustomExceptions.java	Nested runtime exceptions for not-found/import/cycle cases.
db/schema.sql / seed.sql	Manual DDL and sample DML; services do not execute these scripts.
db/recursive_queries.sql	Manual SQL examples, not called from Java.
src/test/java/...	Three Mockito/JUnit unit test classes.
concept-demos/cpp/*	Separate C++ pointer tree demo and Makefile; not app runtime.
concept-demos/python/log_generator.py	Writes synthetic sample files into sample-data.
docs/*.md; project idea/prompt/pptx	Notes and plans; claims should be checked against source.

Entity-to-table map

Entity source	SQL table	Relationship evidence
domain/User.java	users	Referenced by case, note, report, audit.
domain/Asset.java	assets	One asset may have devices.
domain/EndpointDevice.java	endpoint_devices	Many-to-one Asset.
domain/InvestigationCase.java	investigation_cases	Optional assigned User.
domain/EventLog.java	event_logs	Many-to-one case/device; self parentEvent.
domain/KillChainStage.java	kill_chain_stages	Referenced by stage mapping and pattern.
domain/EventStageMapping.java	event_stage_mappings	Many-to-one event and stage.
domain/AttackPattern.java	attack_patterns	Many-to-one stage.
domain/ThreatActorProfile.java	threat_actor_profiles	Reference signature used by DNA analyzer.
domain/RiskAssessment.java	risk_assessments	Many-to-one case.
domain/MitigationStrategy.java	mitigation_strategies	Catalog item.
domain/CaseMitigation.java	case_mitigations	Many-to-one case and catalog item.
domain/Report.java	reports	Many-to-one case and user.
domain/AnalystNote.java	analyst_notes	Many-to-one case and user.
domain/AuditLog.java	audit_logs	Optional many-to-one user.

All 15 repositories extend JpaRepository. Derived query names include findByInvestigationCaseIdOrderByEventTimeAsc; EventLogRepository.filterEvents is the one JPQL method using named parameters. Services use only a subset of the declared repository methods.

5. End-to-end traces and code evidence

A. CSV import

Exact source: src/main/java/com/threattrace/controller/EventController.java:29-37; src/main/java/com/threattrace/service/EventImportService.java:45-101 and 105-148; src/main/java/com/threattrace/service/EventValidationService.java:27-73.

```
EventController.importCsv(caseId,file)
 -> EventImportService.importCsvLogs
 -> caseRepository.findById
 -> CSVReader reads header and rows
 -> validateRawRecord(row, processedCodes)
 -> buildEventLogFromMap(...)
 -> parseTimestamp; find/create device; resolve parent
 -> eventLogRepository.saveAll(validEventsToSave)
 -> ImportResultDTO(caseId,total,valid,errors,messages)
```

Each accepted EventLog has references to its InvestigationCase and EndpointDevice. Device lookup uses hostname; an unknown host can create a device and fallback Asset. Invalid rows are skipped while valid rows persist. DB unique event_code provides another guard. The importer catches many failures and returns normally, so prior accepted rows can commit.

B. Recursive process tree

Exact source: src/main/java/com/threattrace/service/ProcessLineageBuilder.java:25-67; DTO fields are in src/main/java/com/threattrace/dto/DTOs.java:9-69.

```
roots = findByInvestigationCaseIdAndParentEventIsNull(caseId)
for root: buildTreeNodeRecursive(root, 0, new HashSet<>())
  if depth > 25: throw CycleDetectedException
  if event id already in path: throw CycleDetectedException
  create ProcessNodeDTO
  children = findByParentEventId(current.id)
  recurse for each child using a copy of path IDs
```

The path copy permits sibling branches while still identifying a loop to an ancestor. Output is a DTO forest; source data remains event_logs.parent_event_id.

C. Risk arithmetic

Exact source: src/main/java/com/threattrace/service/RiskCalculator.java:35-121; response fields are in src/main/java/com/threattrace/dto/DTOs.java:71-106.

```
asset = max affected asset criticality * 2.5 (max 25)
severity = max event weight: CRITICAL 25, HIGH 18, MEDIUM 10, else 4
stages = distinct mapped stage count * 7.5 (max 52.5)
multiplier = 1.25 when PROCESS cmd/PowerShell child has Word/Excel parent;
             else 1.0
score = min(100, round((asset + severity + stages) * multiplier, 1))
Bands: 75+ CRITICAL; 50+ HIGH; 25+ MEDIUM; else LOW
```

The SQL sample case has max asset criticality 5, critical severity, five stage names, and a matching process ancestry. Current formula: (12.5+25+37.5)*1.25=93.75, shown as 93.8. This is derived from source and sample data, not a live database query. Seeded case score 82.5 is stale relative to this calculation; opening Risk breakdown recalculates.

D. Runtime polymorphism in mitigation ranking

Exact source: src/main/java/com/threattrace/service/MitigationAdvisor.java:21-69; interface: src/main/java/com/threattrace/service/strategy/MitigationRankingStrategy.java; implementations are in the same strategy directory.

```
Map<String, MitigationRankingStrategy> strategyMap
  -> stores RiskReductionStrategy, LowEffortStrategy, BusinessImpactStrategy
GET ...?strategy=LOWEST_EFFORT_FIRST
  -> select interface reference
  -> strategy.calculateScore(item, caseRisk)
  -> save CaseMitigation recommendationScore/rationale
```

The variable is declared as MitigationRankingStrategy while its runtime object is a specific component. Java dispatches to that class implementation. The strategy score is a relative ranking, not a measured intervention result.

E. Notes, audit, and report

Exact source: src/main/java/com/threattrace/service/AnalystNoteService.java:29-49; src/main/java/com/threattrace/service/ReportService.java:52-121; src/main/java/com/threattrace/controller/ReportController.java:25-42.

```
POST notes -> controller passes analyst ID 1
@Transactional service loads case + user
save AnalystNote -> save AuditLog -> return note

POST export -> load case/user/events/tree/risk/DNA
write TEXT / JSON / CSV under reports/
save Report row containing format and file path
```

Report file bytes are on disk; report metadata is in reports. CSV contains a subset of event fields, JSON includes risk/DNA/tree/event count, and text includes case/risk/DNA summaries. There is no export method-level @Transactional annotation.

6. OOP and Java concepts with evidence

Evidence is restricted to checked-in code. The references below name repository paths and exact classes/method ranges; concepts requested but absent are explicitly marked.

Concept	Exact evidence and example	Finding
Classes / objects	domain/EventLog.java:8 and InvestigationCase.java:8; import service creates EventLog instances.	Present: class is a type; object is one event/case instance.
Encapsulation	domain/User.java:8-70; private fields and public getters/setters.	Present. Setters are plain assignments, not business validators.
Constructor overloading	domain/User.java:33-42; no-arg plus parameterized constructor.	Present; compile-time overload selection. No overloaded business methods found.
Inheritance	exception/CustomExceptions.java:5,11,17; nested exceptions extend RuntimeException.	Present. Multilevel app-class inheritance not found.
Interface abstraction	service/strategy/MitigationRankingStrategy.java:5-8.	Present. No Java abstract class found in app.
Overriding/runtime polymorphism	strategy/LowEffortStrategy.java:7-21 and equivalent other strategy classes; advisor map is typed to interface.	Present: implementation method is selected at runtime.
Dependency injection	service/RiskCalculator.java:22-32; EventController.java:20-27.	Spring injects dependencies using constructors. Framework feature built on Java object references.
Access modifiers	domain/EventLog.java private fields; ProcessLineageBuilder.java private recursive helper.	Private fields/helpers and public API methods are present.
static/final	ProcessLineageBuilder.MAX_TREE_DEPTH; EventValidationService.FORMATTERS; DTOs nested static classes.	Present. static final collection reference does not make contents immutable.
Exceptions	Invalid timestamps throw IllegalArgumentException; ResourceNotFound/CycleDetected used; import uses try/catch.	Present; InvalidLogImportException declared but unused. No @ControllerAdvice found.
Collections/generics	List, Map, Set, DTO list fields.	Present.
Lambda / streams	TimelineBuilder stream filters; RiskCalculator max; AttackDNAAnalyzer mapping.	Present.
Recursion / DFS	ProcessLineageBuilder.buildTreeNodeRecursive; MAX_TREE_DEPTH=25.	Present and tested with mocked repositories.
Annotations	@SpringBootApplication, @RestController, @Service, @Entity, @Transactional, @ManyToOne, @Test.	Spring/Jakarta/JUnit metadata; distinguish from Java keywords.
Packages/imports	package com.threattrace.service; imports com.threattrace.domain.* etc.	Present; package names map to directories.
Design pattern	MitigationAdvisor and service/strategy classes.	Strategy pattern present. Avoid claiming all SOLID principles proven.
toString / equals / hashCode override	Search of application Java source.	Not found. Existing concepts note incorrectly claims User.toString().
Multilevel inheritance / abstract class	Search of application Java source.	Not found.
Object serialization	No ObjectOutputStream/Serializable found; Jackson JSON is used.	Java native object serialization not found; JSON is text serialization.
Java concurrency	No Thread/Runnable/executor/background import in core source.	Not found; browser timer is JavaScript polling.

Viva distinctions

- **Overloading:** same constructor name with different argument lists (User constructors), selected at compile time.

- **Overriding:** concrete strategy classes implement interface methods; runtime object behavior handles calculateScore.

- **Inheritance vs association:** exception classes extend RuntimeException. Domain relationships use entity fields and @ManyToOne links.

- **Abstraction / DI:** advisor calls interface contract; Spring constructs concrete beans and injects them.

Existing docs/CONCEPTS_USED_WHERE.md mentions EventValidationService method overloading and User.toString(); neither is present in current source. Study the verified constructor overloading and ranking strategy instead.

7. Database and DBMS guide

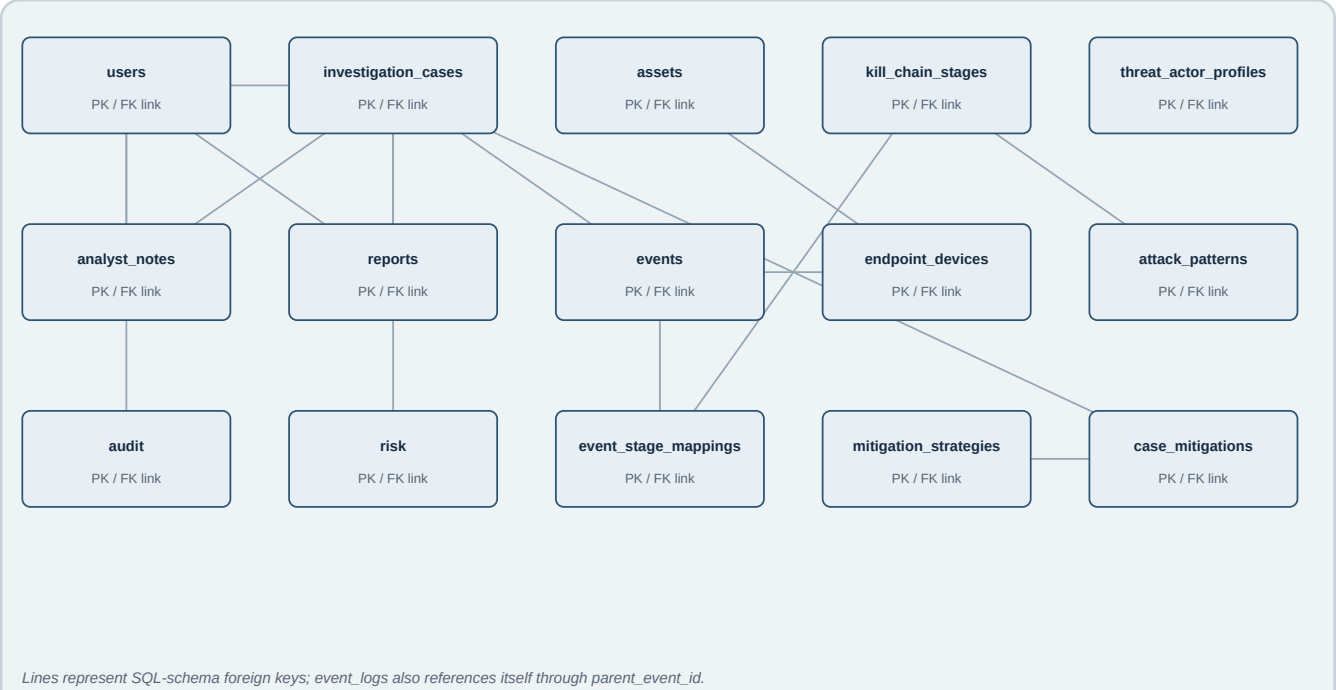
Configured target: MySQL at localhost:3306/threat_trace_db; MySQLDialect; InnoDB in schema/dump. No mysql client was available and no live database query was made. Exact deployed server/version/schema/current rows are **Not verified from source code**. The root threat_trace_db.sql is a saved snapshot only.

Schema-management caution: application.properties sets Hibernate ddl-auto=update. schema.sql is separate manual DDL, not shown being invoked by Java. schema.sql begins by dropping all 15 tables. Never execute it against valuable data without a backup and explicit review. Entity IDs use Long; DDL declares INT; dump has BIGINT. Compare live SHOW CREATE TABLE before claiming exact deployed schema.

15 tables and their columns

Table / primary key	Columns and constraints from resources/db/schema.sql
users (user_id INT)	username VARCHAR(50), password_hash VARCHAR(255), full_name VARCHAR(100), email VARCHAR(100), role VARCHAR(20), created_at TIMESTAMP; username/email unique.
assets (asset_id INT)	asset_name VARCHAR(100), asset_type VARCHAR(50), criticality_score INT CHECK 1..10, owner_team VARCHAR(100), ip_subnet VARCHAR(45), created_at TIMESTAMP.
endpoint_devices (device_id INT)	asset_id INT FK assets, hostname VARCHAR(100) unique, ip_address VARCHAR(45), os_version VARCHAR(100), status VARCHAR(20), created_at TIMESTAMP.
investigation_cases (case_id INT)	case_number VARCHAR(50) unique, title VARCHAR(150), description TEXT, status/severity_level VARCHAR(20), risk_score DOUBLE, assigned_analyst_id INT FK users, created_at/updated_at TIMESTAMP.
event_logs (event_id INT)	event_code VARCHAR(64) unique, case_id/device_id INT FK, event_time DATETIME, event_type VARCHAR(50), severity VARCHAR(20), user_name VARCHAR(100), process_name VARCHAR(255), parent_event_id INT self-FK, target/source VARCHAR(255), description/raw_payload TEXT, created_at TIMESTAMP; indexes case+time, type, parent.
kill_chain_stages (stage_id INT)	stage_name VARCHAR(50) unique, stage_order INT unique, description TEXT.
event_stage_mappings (mapping_id INT)	event_id/stage_id INT FKs, mapping_rule VARCHAR(100), explanation TEXT, mapped_at TIMESTAMP; UNIQUE(event_id,stage_id) in schema.sql.
attack_patterns (pattern_id INT)	pattern_code VARCHAR(50) unique, pattern_name VARCHAR(100), rule_expression TEXT, stage_id INT FK, severity_weight DOUBLE.
threat_actor_profiles (profile_id INT)	actor_codename VARCHAR(100) unique, tactics_summary TEXT, typical_kill_chain_signature VARCHAR(255), risk_indicator_score INT.
risk_assessments (assessment_id INT)	case_id INT FK, total_score DOUBLE, assessment_time TIMESTAMP, algorithm_version VARCHAR(20), factor_breakdown_json TEXT.
mitigation_strategies (mitigation_id INT)	mitigation_code VARCHAR(50) unique, action_name VARCHAR(150), effort_level/business_impact VARCHAR(20), category VARCHAR(50), description TEXT.
case_mitigations (case_mitigation_id INT)	case_id/mitigation_id INT FKs, recommendation_score DOUBLE, status VARCHAR(20), analyst_rationale TEXT, updated_at TIMESTAMP; UNIQUE(case_id,mitigation_id) in schema.sql.
reports (report_id INT)	case_id/generated_by INT FKs, report_format VARCHAR(20), file_path VARCHAR(255), generated_at TIMESTAMP.
analyst_notes (note_id INT)	case_id/analyst_id INT FKs, note_text TEXT, is_false_positive_flag BOOLEAN, created_at TIMESTAMP.
audit_logs (audit_id INT)	user_id INT optional FK, action_type VARCHAR(50), entity_affected VARCHAR(100), timestamp TIMESTAMP, details TEXT.

Entity relationships



Lines represent SQL-schema foreign keys; event_logs also references itself through parent_event_id.

Foreign-key relationships: users -> cases/notes/reports/audit; assets -> devices -> events; cases -> events/risks/notes/reports/case mitigations; event -> parent event and stage mappings; stage -> mappings/patterns; mitigation catalog -> case mitigations. events-to-stages and cases-to-mitigation catalog are many-to-many through junction tables. Java entities expose owning many-to-one fields rather than @OneToMany collections.

Persistence and operations

JPA is the persistence API; **Hibernate** is its ORM implementation; **Spring Data JPA** supplies repository interfaces/derived queries; **JDBC** is the driver connection layer. No raw JDBC connection code found. Java services do not call recursive_queries.sql.

User action	ORM / data effect
Create case	CaseRepository.save -> INSERT investigation_cases.
Import	find/saveAll -> event_logs, sometimes assets/endpoint_devices.
Timeline/tree	SELECT ordered events, roots, and child event rows.
Kill-chain map	SELECT events/stages; saveAll event_stage_mappings transactionally.
Risk	SELECT events/mappings; UPDATE investigation_cases; INSERT risk_assessments except empty-event path.
Mitigations	SELECT catalog/existing rows; INSERT/UPDATE case_mitigations.
Add note	Read case/user; INSERT analyst_notes and audit_logs in @Transactional.
Export/download	Write report file and INSERT reports metadata; download reads filesystem.

Explicit @Transactional methods include import, mapping, risk, mitigation generation, and note creation. Import catches parse errors and returns normally, so accepted rows may commit alongside row errors; do not claim all invalid uploads roll back. ReportService has no method-level transaction.

Normalization and SQL evidence

The schema separates stable host/asset facts from events, stages from event mappings, and mitigation catalog entries from case-specific recommendations. This reduces repetition. A formal 3NF/BCNF certification requires complete functional-dependency analysis; this guide does not claim that table count alone proves normalization. event_logs.raw_payload and risk_assessments.factor_breakdown_json hold serialized text.

resources/db/recursive_queries.sql demonstrates recursive CTE, joins, GROUP BY/HAVING, COUNT/SUM/MAX, subquery, UNION ALL, and order. These are manual DBMS examples, not runtime service SQL. EventLogRepository.filterEvents is JPQL with named parameters; TimelineBuilder uses a derived query and Java stream filtering. Schema indexes: (case_id,event_time), event_type, parent_event_id.

The checked-in dump uses BIGINT IDs and DATETIME(6); resource DDL uses INT and declares delete actions plus unique event-stage and case-mitigation pairs not expressed in JPA annotations. Dump foreign keys and indexes differ. Use the live database, if available, to establish actual deployed facts.

8. Errors, security, and known limitations

Input and error behavior

- **Importer:** requires event_code, event_type, event_time, hostname; checks event-code duplicates and supported timestamp parsers. It does not enforce event/severity enums, all lengths, or full parent referential validation.

- **Exceptions:** ResourceNotFoundException and CycleDetectedException are used. InvalidLogImportException is declared but unused. No @ControllerAdvice/global handler found.

- **UI errors:** requestJSON maps non-2xx to a browser Error. Per-row import validation messages are returned but only counts are shown in the UI.

Security area	Source evidence and limitation
Authentication	User entity has password_hash/role, but no login endpoint, Spring Security config, session, or authorization checks found.
Password handling	No password verification/hash service found. This guide intentionally does not copy credential values from config/SQL.
CORS	Each controller has @CrossOrigin(origins="*"); no origin allowlist is configured.
Database queries	Spring Data methods and JPQL named parameters are used; no string-built SQL in runtime services found. This alone is not an overall security audit.
Configuration secrets	A datasource password is present in application.properties; value redacted here. Move it to env/secret storage and rotate if real. JDBC URL uses useSSL=false and allowPublicKeyRetrieval=true.
Report path	GET /api/reports/download accepts a path string and returns any existing FileSystemResource; source does not constrain it to reports/.
Upload	Limits are 10 MB; UI extension check exists. Server-side type/extension allowlist and malware scan not found.
CSRF / logging	No CSRF security config found; root INFO and com.threattrace DEBUG. No log redaction/retention policy seen.
DOM	Timeline/note rendering uses textContent in observed paths; no full XSS audit was performed.

Important mismatches with project notes

- schema.sql says INT, entity identifiers are Long, and dump snapshot uses BIGINT. Deletion actions and unique pair constraints also differ across SQL artifacts and JPA mappings.

- schema.sql starts with DROP TABLE statements. Setup guide suggests running it manually; do so only against disposable/recreatable data.

- Current source does not implement login/RBAC, case update/close, time filtering in UI, mitigation decision workflow, PDF export, or report preview despite design documents describing goals.

- docs/CONCEPTS_USED_WHERE.md claims User.toString() and EventValidationService method overloading; source shows neither.

- docs/TESTING.md lists MitigationAdvisorTest but no such test class exists. Three test files contain seven @Test methods (3 validation, 2 lineage, 2 risk); they use Mockito, not MySQL integration.

- Some suspicious sample CSV rows are misaligned against their header. Inspect/import a validated sample rather than assuming every generated file is ready.

Database state: direct DB access was not performed. Current schema, server version, and live rows are Not verified from source code. threat_trace_db.sql is a saved snapshot only.

9. OOP viva questions

Question	Short answer
Class vs object?	Class defines a type; an object is an instance. EventLog defines event attributes; import creates instances.
Where is encapsulation?	User.java has private fields and public accessors. Setters are simple assignments, not validators.
Where is constructor overloading?	User has a no-argument JPA constructor and a parameterized constructor. The signature determines compile-time selection.
Where is inheritance?	Nested custom exceptions extend RuntimeException. No event-subclass hierarchy or multilevel application class chain found.
Is MitigationRankingStrategy an abstract class?	No, it is an interface declaring calculateScore and getStrategyName.
Explain runtime polymorphism.	MitigationAdvisor holds the selected implementation through MitigationRankingStrategy and calls calculateScore; the concrete object runs its implementation.
Overloading vs overriding?	Overloading changes parameter lists; overriding/implementing supplies a method body for a parent/interface contract. User constructors overload; strategy classes implement interface methods.
How does dependency injection work?	Spring creates beans and passes repository/service/strategy objects into constructors.
Why DTOs?	DTO classes define response structures, such as recursive process tree, risk breakdown, DNA summary, and import counts. They are not persistent tables.
How is recursion bounded?	ProcessLineageBuilder checks path IDs and maximum depth 25, then throws CycleDetectedException.
Does actor match prove attribution?	No. It is a substring stage-signature similarity lead. It does not prove identity.
Does app have login?	Not in inspected source. There is a user table/entity, but no authentication/authorization flow.
Why @Transactional on notes?	It groups note and audit inserts so a runtime failure escaping can roll back the operation.

10. DBMS viva questions

Question	Short answer
What are table, row, column?	A table is a relation like event_logs; a row is one event; a column is an attribute such as severity.
Primary vs candidate key?	Primary key is the chosen unique identifier; candidate keys are minimal unique identifiers. Several business fields have UNIQUE constraints in schema.sql.
What is a foreign key?	A constraint linking a row to an existing row, such as event_logs.case_id to investigation_cases.case_id.
How is process ancestry represented?	Self-reference event_logs.parent_event_id -> event_logs.event_id.
Where are many-to-many links stored?	event_stage_mappings links events to stages; case_mitigations links cases to catalog mitigation strategies.
What are JDBC/JPA/Hibernate/Spring Data?	JDBC is the driver connection API; JPA is the persistence spec; Hibernate implements ORM; Spring Data provides repository abstractions.
Does app use SQL joins?	recursive_queries.sql has join/aggregate/subquery/union examples, but Java runtime services do not call that file.
What does ddl-auto=update imply?	Hibernate attempts entity-driven schema update at startup; it is separate from manually applying schema.sql.
Does invalid import roll back?	Not necessarily. Valid rows are saved and importer often catches errors/returns normally, so valid data can commit with rejected rows.
What is normalization?	Organizing relations to reduce repeated facts; this project separates devices/assets and catalog/recommendation rows. Formal 3NF/BCNF needs dependency proof.
Is the live database state known?	No. The repo config and dump are visible, but live MySQL state was not queried.

SQL facts to memorize

```
Database target: threat_trace_db
Tables in resource schema.sql: 15
Main event links: case_id, device_id, parent_event_id
Link tables: event_stage_mappings; case_mitigations
Indexes: (case_id,event_time), event_type, parent_event_id
SQL demo: recursive CTE, JOIN, GROUP BY/HAVING, subquery, UNION ALL
```

11. Presentation cheat sheet

Two-minute explanation

“Threat Trace is a Java and Spring Boot web application for investigating endpoint events. The browser calls REST endpoints. The backend creates cases, imports and validates CSV/JSON logs, stores events with case/device/parent references, and displays a timeline. A recursive Java service builds parent-child lineage. Rule-based mappings associate events with Cyber Kill Chain stages. A DNA summary returns event sequence, time gaps, hosts, stages, and a lead similarity. Risk uses asset criticality, maximum event severity, stage coverage, and a specific suspicious process chain multiplier. A strategy interface ranks catalog mitigations. Notes and audit entries persist in the database; TEXT/JSON/CSV reports are written to disk and tracked in a reports table. The code is configured for MySQL, but I will verify the live schema before describing its current state. There is no authentication or authorization enforcement in the source, and similarity does not prove attribution.”

Five-minute demo walkthrough

- **Minute 1:** show ThreatTraceApplication, port 8080, static page, and initial GET /api/cases.

- **Minute 2:** import a validated sample, explain EventValidationService and EventImportService, and identify event_logs/device references.

- **Minute 3:** show timeline and process tree; explain parent_event_id, path Set, and depth 25.

- **Minute 4:** show rule mappings and DNA, then calculate risk using the source formula and sample arithmetic.

- **Minute 5:** change strategy, add a note, export a report, and distinguish MySQL metadata from disk file content.

10 source files to study first

Rank	Path	Reason
1	src/main/java/com/threattrace/ThreatTraceApplication.java	Startup.
2	src/main/resources/application.properties	Port/DB/Hibernate/limits; keep password secret.
3	src/main/resources/static/index.html	User-visible controls.
4	src/main/resources/static/js/app.js	Frontend request chain.
5	src/main/java/com/threattrace/controller/EventController.java	Import/timeline/tree endpoints.
6	src/main/java/com/threattrace/service/EventImportService.java	Parsing and writes.
7	src/main/java/com/threattrace/service/ProcessLineageBuilder.java	Recursion and cycle guard.
8	src/main/java/com/threattrace/service/RiskCalculator.java	Score and persistence.
9	src/main/java/com/threattrace/service/MitigationAdvisor.java	Strategy selection.
10	src/main/resources/db/schema.sql + src/main/java/com/threattrace/domain/EventLog.java	Tables and event relationships.

Avoid these claims

- Do not say roles mean RBAC is active, or say actor similarity identifies the attacker.

- Do not say Java process lineage executes the recursive CTE.

- Do not call recommendation scores measured risk reductions.

- Do not say every failed import rolls back, or that the dump proves the live DB state.

- Do not run schema.sql on data you need. No case close/update, PDF export, or auth endpoint was found.

12. Repository inventory and glossary

Files by role

Group	Included repository files	Use
Build/root	pom.xml; mvnw/mvnw.cmd; .mvn/wrapper/maven-wrapper.properties	Dependencies, Java version, Maven wrapper.
Boot/config	ThreatTraceApplication.java; application.properties	Spring Boot entry and runtime config. Secret value omitted.
Controllers	CaseController, EventController, AnalysisController, ReportController	All HTTP API mappings.
Entities (15)	AnalystNote, Asset, AttackPattern, AuditLog, CaseMitigation, EndpointDevice, EventLog, EventStageMapping, InvestigationCase, KillChainStage, MitigationStrategy, Report, RiskAssessment, ThreatActorProfile, User under src/main/java/com/threattrace/domain/	Persistent entity/table definitions.
DTO/exception	dto/DTOs.java; exception/CustomExceptions.java	Four nested transfer classes and custom exceptions.
Repositories (15)	One JpaRepository interface per entity under repository/.	CRUD and derived query declarations.
Services	Ten service classes plus four service/strategy classes.	Import/analysis/note/report logic.
Frontend	static/index.html; static/css/styles.css; static/js/app.js	Screen structure, styling, fetch/render interactions.
SQL	resources/db/schema.sql, seed.sql, recursive_queries.sql; root threat_trace_db.sql	Manual DDL, seed DML, SQL examples, dump snapshot.
Tests (3)	EventValidationServiceTest, ProcessLineageBuilderTest, RiskCalculatorTest.	Seven JUnit methods, Mockito unit tests.
Sample data (5)	01_normal_events.csv; 02_suspicious_attack_chain.csv; 03_malformed_missing_parents.csv; 04_cyclic_parent_references.csv; 05_sample_logs.json	Inputs; check row/header alignment.
Optional tools	concept-demos/cpp/pointer_tree_demo.cpp + Makefile; concept-demos/python/log_generator.py	Separate C++ demo and synthetic data generator.
Docs/plans	docs/ARCHITECTURE.md, DATABASE_DESIGN.md, CONCEPTS_USED_WHERE.md, SETUP_AND_DEMO.md, TESTING.md; Threat_Trace_Project_Idea.md; Threat_Trace_Antigravity_Prompt.txt; Threat_Trace.pptx	Requirements and explanations, not implementation proof.
Misc	Test.java; concept-demos/cpp/pointer_tree_demo.exe; .gitignore	Empty Java class; prebuilt demo binary; ignore rules.

Glossary

Term	Meaning here
API / endpoint	A URL and HTTP method used by the browser to read/change app data.
Controller	Spring class that receives request data and calls service/repository methods.
Service	Java class containing a workflow or business rule.
Repository	Spring Data interface for finding/saving entity objects.
Entity / ORM	Java object mapped to a relational table by JPA/Hibernate.
DTO	Object shaped for data transfer; not necessarily persisted.
Primary key	Unique row identifier.
Foreign key	Constraint linking a row to another row.
Self-reference	A row in event_logs references another row in event_logs.
Junction table	Table storing links for many-to-many relationships.
Transaction	Grouped database operations with commit/rollback boundaries.
JPQL	Object-oriented query language over entity names/fields.
Derived query	Repository method name Spring Data parses into a query.
Serialization	Converting data to a transport/storage format; here JSON/text/CSV.
CORS	Browser policy for cross-origin web calls; wildcard is configured.
CRUD	Create, Read, Update, Delete; API does not expose all four for cases.
Normalization	Relational decomposition to reduce repeated facts; formal proof needs dependencies.
Recursive CTE	SQL statement referring to prior result rows; only in demo SQL here.
Polymorphism	Calling interface method via shared type while runtime class supplies behavior.
Attribution lead	Similarity prompt for investigation; not proof of identity.

Final rehearsal checklist

- Name the actual entry class, static files, port, and configured DB name without stating the password.

- Trace important user actions through frontend function, endpoint, controller, service, repository/entity, table/file effect, and UI response.

- Distinguish MySQL persistence from report files and from unverified live DB data.
- Explain the actual risk formula and limits of DNA/mitigation scores.
- Distinguish Java recursion from the separate SQL recursive CTE.
- State clearly that authentication/authorization and a global error handler were not found.
- Inspect sample CSV row alignment and query the live database before claiming its current state.

Source version: This guide describes the repository state reviewed for this document. Recheck line references after source changes.